

Documentação da API REST — Monitoramento Ambiental

Visão geral: A API REST do sistema JequiTech é responsável por receber os dados ambientais enviados pelo Gateway, armazená-los no banco de dados MySQL e disponibilizá-los ao Frontend por meio de requisições HTTP.

A API REST foi desenvolvida utilizando as seguintes tecnologias:

- Node.js;
- Express.js;
- MySQL;
- biblioteca mysql2;
- arquitetura em camadas (Routes → Controllers → Models).

Fluxo geral do sistema:

Arduino → Gateway → API REST → Banco de Dados → Frontend

URL base

`http://localhost:3000/api`

Obs: Todos os endpoints apresentados nesta documentação utilizam o prefixo `/api``.

Formato padrão das respostas

Todas as respostas da API seguem, sempre que possível, a estrutura abaixo:

```
``json
{
  "mensagem": "Descrição da operação realizada.",
  "dados": {}
}
```

Formato das datas

Os campos `dataHora` retornados pela API utilizam o padrão **ISO 8601 (UTC)**.

Exemplo:

```
{
  "dataHora": "2026-06-29T02:47:27.000Z"
}
```

1. Usuários

POST /api/usuarios

Cadastra um novo usuário.

Body

```
{  
  "nome": "João Teste",  
  "email": "joao@email.com",  
  "senha": "123456"  
}
```

Resposta 201

```
{  
  "mensagem": "Usuário cadastrado com sucesso.",  
  "dados": {  
    "idUsuario": 1,  
    "nome": "João Teste",  
    "email": "joao@email.com"  
  }  
}
```

Erro 400 — Dados obrigatórios ausentes

```
{  
  "mensagem": "Nome, email e senha são obrigatórios."  
}
```

Erro 500 — Erro interno

```
{  
  "mensagem": "Erro interno ao cadastrar usuário."  
}
```

Função controller: [cadastrarUsuario](#)

Função model: [criarUsuario](#)

GET /api/usuarios

Lista todos os usuários.

Resposta 200

```
{
  "mensagem": "Usuários listados com sucesso.",
  "total": 1,
  "dados": [
    {
      "idUsuario": 1,
      "nome": "João Teste",
      "email": "joao@email.com"
    }
  ]
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao listar usuários."
}
```

Função controller: [buscarUsuarios](#)

Função model: [listarUsuarios](#)

GET /api/usuarios/:id

Busca um usuário pelo ID.

Exemplo:

GET /api/usuarios/1

Resposta 200

```
{
  "mensagem": "Usuário encontrado com sucesso.",
  "dados": {
```

```
"idUsuario": 1,  
"nome": "João Teste",  
"email": "joao@email.com"  
}  
}
```

Erro 404

```
{  
  "mensagem": "Usuário não encontrado."  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao buscar usuário."  
}
```

Função controller: [buscarUsuario](#)

Função model: [buscarUsuarioPorId](#)

PUT /api/usuarios/:id

Atualiza os dados de um usuário.

Body

```
{  
  "nome": "João Atualizado",  
  "email": "joao.atualizado@email.com",  
  "senha": "novaSenha123"  
}
```

Resposta 200

```
{  
  "mensagem": "Usuário atualizado com sucesso."  
}
```

Erro 400

```
{
```

```
"mensagem": "Nome, email e senha são obrigatórios."
}
```

Erro 404

```
{
  "mensagem": "Usuário não encontrado."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao atualizar usuário."
}
```

Função controller: [editarUsuario](#)

Função model: [atualizarUsuario](#)

DELETE /api/usuarios/:id

Remove um usuário.

Resposta 200

```
{
  "mensagem": "Usuário removido com sucesso."
}
```

Erro 404

```
{
  "mensagem": "Usuário não encontrado."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao remover usuário."
}
```

Função controller: `removerUsuario`

Função model: `deletarUsuario`

2. Salas

POST /api/salas

Cadastra uma sala monitorada.

Body

```
{
  "nome": "Laboratório de Informática",
  "descricao": "Sala monitorada pelo Arduino",
  "tempMin": 20,
  "tempMax": 30,
  "umidadeMin": 40,
  "umidadeMax": 70,
  "luxMin": 100,
  "luxMax": 800,
  "idUsuario": 1
}
```

Resposta 201

```
{
  "mensagem": "Sala cadastrada com sucesso.",
  "dados": {
    "idSala": 1,
    "nome": "Laboratório de Informática",
    "descricao": "Sala monitorada pelo Arduino",
    "tempMin": 20,
    "tempMax": 30,
    "umidadeMin": 40,
    "umidadeMax": 70,
    "luxMin": 100,
    "luxMax": 800,
    "idUsuario": 1
  }
}
```

Erro 400

```
{  
  "mensagem": "Nome da sala e idUsuario são obrigatórios."  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao cadastrar sala."  
}
```

Função controller: [cadastrarSala](#)

Função model: [criarSala](#)

GET /api/salas

Lista todas as salas.

Resposta 200

```
{  
  "mensagem": "Salas listadas com sucesso.",  
  "total": 1,  
  "dados": [  
    {  
      "idSala": 1,  
      "nome": "Laboratório de Informática",  
      "descricao": "Sala monitorada pelo Arduino",  
      "tempMin": "20.00",  
      "tempMax": "30.00",  
      "umidadeMin": "40.00",  
      "umidadeMax": "70.00",  
      "luxMin": "100.00",  
      "luxMax": "800.00",  
      "idUsuario": 1  
    }  
  ]  
}
```

Erro 500

```
{
```

```
"mensagem": "Erro interno ao listar salas."
}
```

Função controller: [buscarSalas](#)

Função model: [listarSalas](#)

GET /api/salas/:id

Busca uma sala pelo ID.

Resposta 200

```
{
  "mensagem": "Sala encontrada com sucesso.",
  "dados": {
    "idSala": 1,
    "nome": "Laboratório de Informática",
    "descricao": "Sala monitorada pelo Arduino",
    "tempMin": "20.00",
    "tempMax": "30.00",
    "umidadeMin": "40.00",
    "umidadeMax": "70.00",
    "luxMin": "100.00",
    "luxMax": "800.00",
    "idUsuario": 1
  }
}
```

Erro 404

```
{
  "mensagem": "Sala não encontrada."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao buscar sala."
}
```


Função controller: `buscarSala`
Função model: `buscarSalaPorId`

PUT /api/salas/:id

Atualiza uma sala.

Body

```
{  
  "nome": "Laboratório Atualizado",  
  "descricao": "Sala principal do projeto",  
  "tempMin": 18,  
  "tempMax": 32,  
  "umidadeMin": 35,  
  "umidadeMax": 75,  
  "luxMin": 80,  
  "luxMax": 900,  
  "idUserario": 1  
}
```

Resposta 200

```
{  
  "mensagem": "Sala atualizada com sucesso."  
}
```

Erro 400

```
{  
  "mensagem": "Nome da sala e idUsuario são obrigatórios."  
}
```

Erro 404

```
{  
  "mensagem": "Sala não encontrada."  
}
```

Erro 500

```
{
```

```
"mensagem": "Erro interno ao atualizar sala."
}
```

Função controller: [editarSala](#)

Função model: [atualizarSala](#)

DELETE /api/salas/:id

Remove uma sala.

Resposta 200

```
{
  "mensagem": "Sala removida com sucesso."
}
```

Erro 404

```
{
  "mensagem": "Sala não encontrada."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao remover sala."
}
```

Função controller: [removerSala](#)

Função model: [deletarSala](#)

3. Sensores

POST /api/sensores

Cadastra um sensor.

Body

```
{
```

```
"tipo": "temperatura",
"status": "ativo",
"idSala": 1
}
```

Resposta 201

```
{
  "mensagem": "Sensor cadastrado com sucesso.",
  "dados": {
    "idSensor": 1,
    "tipo": "temperatura",
    "status": "ativo",
    "idSala": 1
  }
}
```

Erro 400

```
{
  "mensagem": "Tipo, status e idSala são obrigatórios."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao cadastrar sensor."
}
```

Função controller: [cadastrarSensor](#)

Função model: [criarSensor](#)

GET /api/sensores

Lista todos os sensores.

Resposta 200

```
{
  "mensagem": "Sensores listados com sucesso.",
  "total": 3,
  "dados": [
```

```
{
  "idSensor": 1,
  "tipo": "temperatura",
  "status": "ativo",
  "idSala": 1
},
{
  "idSensor": 2,
  "tipo": "umidade",
  "status": "ativo",
  "idSala": 1
},
{
  "idSensor": 3,
  "tipo": "luminosidade",
  "status": "ativo",
  "idSala": 1
}
]
```

Erro 500

```
{
  "mensagem": "Erro interno ao listar sensores."
}
```

Função controller: [buscarSensores](#)

Função model: [listarSensores](#)

GET /api/sensores/:id

Busca um sensor pelo ID.

Resposta 200

```
{
  "mensagem": "Sensor encontrado com sucesso.",
  "dados": {
    "idSensor": 1,
    "tipo": "temperatura",
    "status": "ativo",
```

```
    "idSala": 1
  }
}
```

Erro 404

```
{
  "mensagem": "Sensor não encontrado."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao buscar sensor."
}
```

Função controller: `buscarSensor`

Função model: `buscarSensorPorId`

PUT /api/sensores/:id

Atualiza um sensor.

Body

```
{
  "tipo": "temperatura",
  "status": "inativo",
  "idSala": 1
}
```

Resposta 200

```
{
  "mensagem": "Sensor atualizado com sucesso."
}
```

Erro 400

```
{
  "mensagem": "Tipo, status e idSala são obrigatórios."
}
```

Erro 404

```
{  
  "mensagem": "Sensor não encontrado."  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao atualizar sensor."  
}
```

Função controller: `editarSensor`

Função model: `atualizarSensor`

DELETE /api/sensores/:id

Remove um sensor.

Resposta 200

```
{  
  "mensagem": "Sensor removido com sucesso."  
}
```

Erro 404

```
{  
  "mensagem": "Sensor não encontrado."  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao remover sensor."  
}
```

Função controller: `removerSensor`

Função model: `deletarSensor`

4. Leituras

POST /api/leituras

Recebe os dados enviados pelo Gateway.

Body

```
{
  "dispositivo": "arduino-uno-01",
  "numeroLeitura": 1,
  "temperatura": 25.6,
  "umidade": 58.4,
  "luminosidadeLux": 420.5,
  "tempoMillis": 2000
}
```

Resposta 201

```
{
  "mensagem": "Leitura salva com sucesso.",
  "dados": {
    "dispositivo": "arduino-uno-01",
    "numeroLeitura": 1,
    "temperatura": 25.6,
    "umidade": 58.4,
    "luminosidadeLux": 420.5,
    "tempoMillis": 2000,
    "dataHora": "2026-06-29T02:47:27.000Z"
  }
}
```

Erro 400

```
{
  "mensagem": "Dados inválidos. Verifique o formato enviado pelo gateway."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao salvar leitura."
}
```

```
}
```

Função controller: `cadastrarLeitura`

Função model: `salvarLeitura`

GET /api/leituras

Lista o histórico de leituras.

Resposta 200

```
{
  "mensagem": "Leituras listadas com sucesso.",
  "total": 3,
  "dados": [
    {
      "idLeitura": 1,
      "sensor": "temperatura",
      "valor": 25.6,
      "dataHora": "2026-06-29T02:47:27.000Z"
    },
    {
      "idLeitura": 2,
      "sensor": "umidade",
      "valor": 58.4,
      "dataHora": "2026-06-29T02:47:27.000Z"
    },
    {
      "idLeitura": 3,
      "sensor": "luminosidade",
      "valor": 420.5,
      "dataHora": "2026-06-29T02:47:27.000Z"
    }
  ]
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao listar leituras."
}
```


Função controller: `buscarLeituras`

Função model: `listarLeituras`

GET /api/leituras/ultima

Retorna a última leitura consolidada.

Resposta 200

```
{
  "mensagem": "Última leitura consultada com sucesso.",
  "dados": {
    "dataHora": "2026-06-29T02:47:27.000Z",
    "temperatura": 25.6,
    "umidade": 58.4,
    "luminosidadeLux": 420.5
  }
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao buscar última leitura."
}
```

Função controller: `buscarUltima`

Função model: `buscarUltimaLeitura`

5. Alertas

POST /api/alertas

Cadastra um alerta.

Body

```
{
  "tipo": "Temperatura acima do limite",
  "idSensor": 1
}
```

Resposta 201

```
{
  "mensagem": "Alerta cadastrado com sucesso.",
  "dados": {
    "idAlerta": 1,
    "tipo": "Temperatura acima do limite",
    "dataHora": "2026-06-29T02:47:27.000Z",
    "idSensor": 1
  }
}
```

Erro 400

```
{
  "mensagem": "Tipo do alerta e idSensor são obrigatórios."
}
```

Erro 500

```
{
  "mensagem": "Erro interno ao cadastrar alerta."
}
```

Função controller: [cadastrarAlerta](#)

Função model: [criarAlerta](#)

GET /api/alertas

Lista todos os alertas.

Resposta 200

```
{
  "mensagem": "Alertas listados com sucesso.",
  "total": 1,
  "dados": [
    {
      "idAlerta": 1,
      "tipo": "Temperatura acima do limite",
      "dataHora": "2026-06-29T02:47:27.000Z",
      "idSensor": 1,
      "tipoSensor": "temperatura"
    }
  ]
}
```

```
}  
]  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao listar alertas."  
}
```

Função controller: [buscarAlertas](#)

Função model: [listarAlertas](#)

GET /api/alertas/:id

Busca um alerta pelo ID.

Resposta 200

```
{  
  "mensagem": "Alerta encontrado com sucesso.",  
  "dados": {  
    "idAlerta": 1,  
    "tipo": "Temperatura acima do limite",  
    "dataHora": "2026-06-29T02:47:27.000Z",  
    "idSensor": 1,  
    "tipoSensor": "temperatura"  
  }  
}
```

Erro 404

```
{  
  "mensagem": "Alerta não encontrado."  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao buscar alerta."  
}
```

Função controller: `buscarAlerta`
Função model: `buscarAlertaPorId`

DELETE /api/alertas/:id

Remove um alerta.

Resposta 200

```
{  
  "mensagem": "Alerta removido com sucesso."  
}
```

Erro 404

```
{  
  "mensagem": "Alerta não encontrado."  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao remover alerta."  
}
```

Função controller: `removerAlerta`
Função model: `deletarAlerta`

6. Dashboard

GET /api/dashboard

Retorna os dados resumidos para o painel.

Resposta 200

```
{  
  "mensagem": "Dados do dashboard consultados com sucesso.",  
  "dados": {  
    "temperatura": 25.6,  
    "umidade": 58.4,
```

```
"luminosidadeLux": 420.5,  
"status": "normal",  
"dataHora": "2026-06-29T02:47:27.000Z"  
}  
}
```

Resposta 200 — Sem leituras

```
{  
  "mensagem": "Nenhuma leitura encontrada.",  
  "dados": null  
}
```

Erro 500

```
{  
  "mensagem": "Erro interno ao buscar dados do dashboard."  
}
```

Função controller: [buscarDashboard](#)

Função model: [buscarDadosDashboard](#)

7. Códigos HTTP utilizados

200 OK — Requisição executada com sucesso.

201 Created — Registro criado com sucesso.

400 Bad Request — Dados inválidos ou incompletos.

404 Not Found — Registro não encontrado.

500 Internal Server Error — Erro interno da API ou do banco de dados.

8. Tratamentos de erro

A API trata os seguintes erros principais:

- ausência de campos obrigatórios;
- tentativa de buscar registro inexistente;
- tentativa de editar registro inexistente;
- tentativa de remover registro inexistente;
- erro interno de conexão ou execução SQL;

- JSON inválido vindo do Gateway.
-

9. Integração

Gateway → API

O Gateway envia dados para:

POST /api/leituras

Frontend → API

O Frontend consome principalmente:

GET /api/dashboard

GET /api/leituras

GET /api/leituras/ultima